HCM ↔ SCM — API & Webhook Requirements

1. What changes and why

Today SCM keeps a **local mirror** of HCM. Nine tables are pulled on a schedule and copied row-for-row:

Mirror table	What it holds
`hcm_employees`	every person; **this is SCM's identity/login table** — SCM has no `users` table
`hcm_positions`	the org chart: seat, its parent seat, its rank (`role_id`), its project
`hcm_position_assignments`	which person sits in which seat
`hcm_departments`	departments
`hcm_sections`	sections
`hcm_offices`	offices / locations
`hcm_position_levels`	seniority bands
`hcm_facilities`	facilities
`hcm_facility_types`	facility masters
(+ `mst_projects`)	projects, HCM-owned, SCM-read-only

We want to delete all of it. Going forward SCM stores **zero HCM rows** and asks HCM live, over HTTP, for anything about a person, a seat or an office.

Two consequences drive everything in this document:

1. **HCM becomes a hard dependency for login and authorization.** If HCM is down, nobody can log in to SCM and no purchase order can be approved. So the read APIs below need real latency and availability numbers, not best-effort.
2. **Polling cannot replace the mirror.** The mirror hid a lot of graph work — climbing a reporting chain, listing a subtree, finding who sits in a rank. Those must become **purpose-built endpoints** (§5.C, §5.D). A generic `GET /employees?page=1` list API will not work: one approval submit would become ~15 sequential round-trips.

Concern, stated once and then set aside: removing the mirror trades a stale-data problem for an availability problem. We accept it provided §8 (SLA) and §7 (webhooks) are met, plus SCM keeping a short-lived **in-process cache** (seconds–minutes, RAM only, never written to the database). That cache is not a mirror and needs no HCM work beyond the `ETag` / `Cache-Control` headers requested in §4.6.

What SCM still stores locally (not HCM data, do not send it)

- `sys_user_credentials` — SCM's own password hashes, until HCM exposes a login API (§5.A.4).
- **Snapshot columns on documents** — when an indent is raised, SCM freezes `raised_by_name`, `raised_by_code`, `department_name`, `office_name` onto the document. That is a historical record, not a cache: it must never change when the person later transfers or leaves. Those columns stay.
- `rbac_*`, `wf_*` — SCM's own roles, permissions and approval ladders. These *reference* HCM ids (`role_id`, `position_id`, `employee_id`) as plain integers.

---

2. Non-negotiable data rules

Never send these fields, at any nesting depth, on any endpoint or webhook:

bank_details, bank_account, account_number, ifsc, ifsc_code,
salary, salary_details, ctc, gross_salary, net_salary,
epfo, pf_number, uan, esi_number,
health, health_details, medical, blood_group,
pan_number, pan, aadhaar_number, aadhaar,
dob, date_of_birth, gender, photo

SCM reads none of it and will drop it on arrival. A supply-chain database must not hold national ID or payroll data. If a field on this list is required for a genuine SCM feature later, it gets its own request and its own review.

Ids are permanent. `employee.id`, `position.id`, `office.id`, `department.id`, `project.id`, `role.id` are the join keys stamped onto SCM purchase orders, GRNs and approval records going back years. They must never be reused for a different entity and never renumbered. If HCM migrates its own storage, ids survive the migration or HCM supplies an id-remap file before cutover.

Nothing is ever hard-deleted. A person who leaves becomes `status: "inactive"`. A seat that is abolished becomes inactive. `GET /employees/{id}` for an ex-employee

must keep returning 200 with the inactive record forever — SCM has to render "Raised by: R. Kumar" on a 2024 indent whose author left in 2025.

---

3. Interface conventions

3.1 Base URL and versioning

https:///api/scm/v1/

A versioned prefix, separate from HCM's internal UI API. Breaking changes ship as `v2`; `v1` stays up for at least 6 months after `v2` is announced.

3.2 Authentication

- Header `X-API-Key: <key>` — machine-to-machine, issued to SCM, rotatable without a code change (SCM reads it from `HCM_API_KEY` in its environment).
- Keys are scoped: the SCM key can read only the entities in this document, and can write nothing.
- HCM must support **two live keys at once** during rotation (old + new valid for a 24h overlap).
- Requests carry `X-Request-Id` (uuid) from SCM; HCM echoes it back in the response and includes it in its own logs so an incident can be traced across both systems.

3.3 Response envelope

Single object:

```
{ "data": { ... } }
```

Collection:

```
{
  "data": [ ... ],
  "page": { "limit": 100, "offset": 0, "total": 4173, "next": "https://..
}
```

`next` is `null` on the last page. `total` is required (SCM paginates its own screens off it).

3.4 Errors

One shape, always:

```
{ "error": { "code": "NOT_FOUND", "message": "No employee with id 99999",
```

HTTP	code	When
400	`BAD_REQUEST`	malformed parameter
401	`UNAUTHENTICATED`	missing/invalid API key
403	`FORBIDDEN`	key not scoped to this entity
404	`NOT_FOUND`	unknown id
409	`CONFLICT`	e.g. credential verify on a locked account
422	`VALIDATION_ERROR`	valid syntax, impossible request
429	`RATE_LIMITED`	+ `Retry-After` header
500	`INTERNAL_ERROR`	HCM fault
503	`UNAVAILABLE`	HCM in maintenance; + `Retry-After`

Never return 200 with an error body. Never return HTML.

3.5 Types

- Timestamps: RFC 3339 UTC, `2026-08-05T11:04:07Z`. Dates: `2026-08-05`.
- Ids: JSON integers (not strings).
- `status`: the string `"active"` or `"inactive"` (SCM maps to a boolean).
- Booleans are real booleans, not `"Y"` / `0` / `"true"`.
- Absent value = `null`, not `""` and not omitted.

3.6 Caching

Every GET returns `ETag` and honours `If-None-Match` → `304 Not Modified`. `Cache-Control: private, max-age=<n>` with a sensible `n` per §8 table. This is what lets SCM hold a small RAM cache and still be correct.

---

4. Common object shapes

Referenced by the endpoints in §5. Every object includes `status`.

4.1 `Employee`

```
{
  "id": 10432,
  "employee_code": "TIR-0432",
  "name": "R. Kumar",
  "email": "r.kumar@example.com",
  "phone": "9876543210",
  "status": "active",
  "date_of_joining": "2021-06-14",
  "date_of_exit": null,
  "office":     { "id": 12, "code": "ATP", "name": "Anantapur" },
  "department": { "id": 4,  "code": "NW",  "name": "Network" },
  "section":    { "id": 19, "code": "NW-FLD", "name": "Field Ops" },
  "project":    { "id": 7,  "code": "PRJ-APSFL", "name": "APSFL Phase 2"
  "primary_position": { "id": 880, "name": "District Manager - Anantapur"
  "reporting_manager": { "id": 10077, "name": "S. Prasad" },
  "updated_at": "2026-08-01T09:12:00Z"
}
```

`primary_position` inline is **required** — SCM needs a person's seat on every login, and a second round-trip per login is the difference between a 200 ms and a 600 ms sign-in.

4.2 `Position` (a seat in the org chart)

```
{
  "id": 880,
  "code": "POS-880",
  "name": "District Manager - Anantapur",
  "status": "active",
  "parent_position_id": 654,
  "level":      { "id": 5, "code": "L5", "name": "Manager", "rank": 5 },
  "role":       { "id": 31, "name": "District Manager" },
  "job_name":   "District Manager",
  "office_level": "District",
  "department": { "id": 4, "name": "Network" },
  "office":     { "id": 12, "name": "Anantapur" },
  "project":    { "id": 7, "name": "APSFL Phase 2" },
```

```
  "assigned_employees": [ { "id": 10432, "name": "R. Kumar", "employee_co
  "holder_count": 1,
  "updated_at": "2026-07-30T06:00:00Z"
}
```

`office_level` **must be a first-class field.** SCM reads it today by digging into the position's raw HCM payload (`raw.office_level`) to seed row-level scoping — it decides whether a login sees only their own documents, their office's, their subtree's, their project's, or everything. Reading it out of an untyped blob is fragile; it needs to be a documented field with a documented value set (`Head Office | State | Region | District | Mandal | …`). Same for `raw.project_name` , which SCM currently string-matches back to a project id: send `project` as an object with an id (above) and that hack disappears.

`code` **format.** SCM parses the position code as `<PROFILE-KEY>@<suffix>` — it splits on `@` and uses the left side to group seats into job profiles. If that convention is accidental rather than guaranteed, say so and SCM will stop relying on it; if it is intentional, document it here as part of the contract.

`role` **is the rung SCM's approval ladders are built on.** It is HCM's *rank* — "District Manager", "Regional Manager", "COO" — not a permission and not a person. `role_id` must be stable and must be present on every active position; a position with `role_id: null` is invisible to approval routing and will strand documents.

`parent_position_id` **is the single most important field in this integration.** SCM climbs it at approval time to find who signs off. Requirements:

- It points at a **position id**, never an employee id.
- The graph must be **acyclic**. The current mirror contains cycles (a seat whose ancestor chain loops back to itself); SCM defends with a seen-set, but HCM must validate on write and reject a change that would create a cycle.
- A root seat has `parent_position_id: null` .

4.3 PositionAssignment

```
{
  "id": 4501,
  "employee": { "id": 10432, "name": "R. Kumar", "employee_code": "TIR-04
  "position": { "id": 880, "name": "District Manager - Anantapur" },
  "department": { "id": 4, "name": "Network" },
  "office": { "id": 12, "name": "Anantapur" },
```

```
    "is_primary": true,
    "valid_from": "2024-04-01",
    "valid_to": null,
    "status": "active"
  }
```

A person can hold more than one seat (SCM has a position switcher); exactly one must be `is_primary: true`.

4.4 Office

```
{ "id": 12, "code": "ATP", "name": "Anantapur", "office_type": "District"
  "level": { "id": 3, "name": "District", "rank": 3 },
  "parent_office_id": 2, "address": "…", "city": "Anantapur", "state": "A
  "status": "active" }
```

`parent_office_id` and `level.rank` are new asks — SCM's row-level scoping ("this District Manager sees only Anantapur's documents") currently infers the hierarchy and gets it wrong at the edges.

4.5 Department / Section / PositionLevel / Role / Project / Facility

```
{ "id": 4,  "code": "NW", "name": "Network", "office_id": 12, "head_emplo
{ "id": 19, "code": "NW-FLD", "name": "Field Ops", "department_id": 4, "s
{ "id": 5,  "code": "L5", "name": "Manager", "rank": 5, "status": "active
{ "id": 31, "name": "District Manager", "rank": 5, "status": "active" }
{ "id": 7,  "code": "PRJ-APSFL", "name": "APSFL Phase 2", "start_date": "
  "offices": [ { "id": 12, "name": "Anantapur" } ], "status": "active" }
{ "id": 88, "code": "FAC-88", "name": "Anantapur Central Store", "facilit
  "office_id": 12, "address": "…", "status": "active" }
```

`head_employee_id` on a department is currently always absent in the live feed — SCM needs it populated (department-head approval routing depends on it).

---

5. Read APIs

Each entry gives: purpose → who calls it → input → output → how often SCM will call it.

A. Identity, login and session

A.1 `GET /employees/{id}`

Resolve the JWT subject on **every authenticated request**.

Input	path `id`
Output	`{ "data": Employee }` (§4.1)
404	unknown id
Frequency	very high — every API call SCM serves. Cacheable 60 s.

A.2 `GET /employees/lookup?username=<value>`

Login step 1. `username` may be an employee id, an `employee_code`, or an email; HCM matches any of the three (SCM's login box accepts all three today).

Input	query `username` (required)
Output	`{ "data": Employee }`, or 404
Note	must **not** reveal whether a username exists in a timing-obvious way
Frequency	one per login attempt

A.2b `GET /employees/by-code/{employee_code}` ★

`employee_code` ("TIR-0432") is what humans type, print on documents and paste into a support ticket — SCM users do not know numeric HCM ids. A.2 handles it for login; this is the general-purpose version.

Input	path `employee_code` (exact match, case-insensitive)
Output	`{ "data": Employee }` — identical shape to A.1
404	unknown code

Employee code must be accepted everywhere an id is, not just here:

Endpoint	id form	code form
A.1 / A.2b	`GET /employees/{id}`	`GET /employees/by-code/{code}`

Endpoint	id form	code form
B.1 batch	`{"ids":[…]}`	`{"employee_codes":[…]}` — either key, or both
B.3 resolve-names	`{"ids":[…]}`	`{"employee_codes":[…]}`, keyed by code in the response
C.4 / C.6	`/employees/{id}/positions`, `/reports`	accept `?employee_code=` instead of the path id
I.1 list	`?ids=`	`?employee_codes=TIR-0432,TIR-0077`

Codes must be **unique and never reused**, including across an employee who left and a new joiner. If HCM cannot guarantee that, say so — SCM will then treat the code as display-only and force every lookup through the numeric id.

A.3 `GET /employees/{id}/context` ★ required

The one call SCM makes right after login to build the whole session. Replaces 5–8 round-trips.

Input	path `id`; optional query `position_id` (when the user switched seats)
Frequency	one per login + one per position switch

```
{
  "data": {
    "employee": { ...Employee... },
    "positions": [
      { ...Position..., "assignment": { "id": 4501, "is_primary": true, "
    ],
    "active_position": { ...Position... },
    "ancestors": [ { ...Position... }, { ...Position... } ],
    "office_path": [ { "id": 1, "name": "Head Office" }, { "id": 2, "name
    "direct_reports_count": 14
  }
}
```

`ancestors` = the reporting chain from this person's seat up to the root, ordered nearest-first. `office_path` = root-to-leaf office hierarchy, used for row scoping.

A.4 `POST /auth/verify-credentials` — *optional, phase 2*

If HCM ever owns SCM's passwords, this is the shape SCM will call. Until it exists, SCM keeps `sys_user_credentials` and its own JWT.

```
POST /auth/verify-credentials
{ "username": "TIR-0432", "password": "…" }
→ 200 { "data": { "employee_id": 10432, "verified": true } }
→ 401 { "error": { "code": "UNAUTHENTICATED", ... } }
→ 409 { "error": { "code": "ACCOUNT_LOCKED", ... } }
```

Rate-limited by HCM per username. SCM's JWT, `/api/auth/me` and `require_permission` are unchanged either way.

B. Bulk reads (the N+1 killers)

Without these, removing the mirror makes SCM unusably slow. They are not optional.

B.1 `POST /employees/batch`

```
POST /employees/batch
{ "ids": [10432, 10077, 10981], "fields":
["id","name","employee_code","status","office","project"] }
→ { "data": [ Employee, ... ], "missing": [ 10981 ] }
```

- Up to **500 ids** per call.
- `fields` optional; when given, HCM returns only those keys (SCM often needs just id+name for a picker).
- `missing` lists ids HCM does not know — never 404 the whole batch for one bad id.
- Frequency: several per list screen render.

B.2 `POST /positions/batch`

Same contract, for position ids. Up to 500. Used by ladder rendering and scope checks.

B.3 `POST /employees/resolve-names`

The cheapest possible call — ids to display names, nothing else.

```
POST /employees/resolve-names
{ "ids": [10432, 10077] }
→ { "data": { "10432": { "name": "R. Kumar", "employee_code": "TIR-0432",
"status": "active" },
```

"10077": { "name": "S. Prasad", "employee_code": "TIR-0077", "status": "inactive" } } }

Must answer in under 100 ms for 200 ids. This one endpoint carries most of SCM's read traffic.

C. Org-graph traversal ★ the reason a plain list API is not enough

SCM's approval engine, at **submit time**, climbs the raiser's `parent_position_id` chain and keeps the ancestors whose rank is switched on in the configured ladder. Over the mirror that is one recursive walk in SQL. Over a REST list API it is one HTTP call per hop, serially — unacceptable. HCM must do the walk.

C.1 `GET /positions/{id}/ancestors`

Purpose	the reporting chain above a seat — **used on every document submitted for approval**
Input	path `id` ; optional `max_depth` (default 20), optional `include_holders=true`
Output	ordered array, nearest ancestor first
Latency	**p95 < 200 ms** — a user is watching a spinner


```
{
  "data": [
    { "id": 654, "name": "Regional Manager - Rayalaseema", "role": { "id"
      "parent_position_id": 301, "status": "active", "depth": 1,
      "holders": [ { "id": 10077, "name": "S. Prasad", "employee_code": "
    { "id": 301, "name": "State Project Head - AP", "role": { "id": 12, "
      "parent_position_id": 100, "status": "active", "depth": 2, "holders
    { "id": 100, "name": "COO", "role": { "id": 3, "name": "COO" },
      "parent_position_id": null, "status": "active", "depth": 3, "holder
  ]
}
```


`include_holders=true` must be supported — otherwise SCM does one extra call per rung and we are back to N+1. Inactive seats are **included** with `status:"inactive"` so SCM can skip them and log why, rather than silently truncating a chain.
If the walk detects a cycle, return the chain up to the repeat plus `"warning": "cycle_detected_at_position_id: 654"`.

C.2 `GET /positions/{id}/descendants`

Purpose	row-level scoping: "a manager sees their own seat and everything under it"
Input	path `id` ; `max_depth` ; `status=active
Output	with `ids_only=true` : `{ "data": [880, 881, 903, ...] }` — can be thousands, so this form matters
Frequency	every list screen for a SUBTREE-scoped user. Cacheable 5 min.

C.3 `GET /positions/{id}/holders`

Who currently sits in a seat.

```
GET /positions/880/holders?as_of=2026-08-05
→ { "data": [ { "employee": {...}, "assignment": {...} } ] }
```

`as_of` optional — SCM occasionally needs "who held this seat when the PO was approved". Empty array is a valid answer (a vacant seat), and SCM will skip that rung.

C.4 `GET /employees/{id}/positions`

All seats a person holds, primary flagged. Powers SCM's position switcher.

C.5 `GET /employees/{id}/manager-chain`

Convenience wrapper on C.1 keyed by employee instead of position — same response shape, saves a lookup. Optional if C.1 exists.

C.6 `GET /employees/{id}/reports` ★ the downward direction

C.1/C.5 answer "who is above this person". This answers **"who is below"** — and SCM needs it just as often: a manager's dashboard, "documents raised by my team", bulk notification to a team, and reassigning work when someone resigns.

```
GET /employees/10077/reports?depth=direct        → people in the seats
directly under theirs
GET /employees/10077/reports?depth=all           → the whole subtree,
every level
GET /employees/10077/reports?depth=all&ids_only=true
GET /employees/10077/reports?
depth=all&status=active&office_id=&project_id=&limit=&offset=
```

```
{
  "data": [
    { "employee": { "id": 10432, "name": "R. Kumar", "employee_code": "TI
      "position": { "id": 880, "name": "District Manager - Anantapur", "r
      "depth": 1, "via_position_id": 654 }
  ],
  "page": { "total": 47 }
}
```

- `depth=direct` is one hop; `depth=all` walks the whole subtree; `depth=3` caps it.
- `ids_only=true` returns a bare employee-id array — this is the form SCM's list screens use to filter their own documents (§I.3), so it must be uncapped.
- A vacant seat in the middle of the tree does **not** stop the walk — the people under it are still reports. Add `include_vacant_positions=true` to also list the empty seats, which SCM's admin screen shows as gaps.
- If a person holds two seats, reports are the union of both subtrees, deduplicated.

C.7 `GET /positions/{id}/children`

The one-hop version of C.2, for rendering an org chart lazily one level at a time rather than pulling a whole subtree.

D. Rank / role queries (approval-ladder configuration)

SCM's "Approvals Config" screen lets an admin build a ladder for a project by switching **ranks** on and off. It needs to list the ranks that actually exist in a project and who fills them.

D.1 `GET /roles`

Every org rank. `{ "data": [ Role ], "page": {...} }`. Cacheable 1 h.
SCM mirrors these into `rbac_roles` **by id only** so permissions can target a rank.

D.2 `GET /roles?project_id=7`

Only the ranks held by people on that project. This is what fills the ladder builder, and it must be one call — today SCM derives it from three joins.

```
{ "data": [ { "id": 31, "name": "District Manager", "rank": 5,
              "position_count": 13, "holder_count": 13 } ] }
```

D.3 `GET /roles/{role_id}/positions?project_id=7&include_holders=true`

Which seats carry a rank, and who is in them. Drives the "who will approve" preview.

D.4 `GET /positions?project_id=&office_id=&department_id=&role_id=&status=&q=&limit=&offset=`

The general position list, filterable on every axis SCM's screens filter on.
`q` searches name and code.

E. Project-scoped reads

E.1 `GET /projects`

All projects. Add `include_headcount=true` → each project carries `"headcount": 412`. SCM's ladder screen orders projects by headcount and currently computes that with a `GROUP BY` over the mirror.

E.2 `GET /projects/{id}`

One project, with `offices[]`.

E.3 `GET /projects/{id}/employees?status=active&limit=&offset=`

People on a project.

E.4 `GET /projects/{id}/positions?ids_only=true`

The seats held by that project's people — the sub-select behind every ladder query.

Important: a person's project lives on the **employee**, not on the position. `hcm_positions.project_id` is empty in the live feed today. Either populate it or confirm in writing that employee-level project is the only source; SCM routes multi-region approvals off this and a wrong answer sends a purchase order to the wrong region's manager.

F. Reference lists

Plain paginated lists, all supporting `?status=active|inactive|all`, `?q=`, `?updated_since=`, `?limit=`, `?offset=`. Long cache lives (1 h) — these barely change.

Endpoint	Notes
`GET /offices`	+ `GET /offices/{id}`, `GET /offices/{id}/descendants` for office-tree scoping
`GET /departments`	must include `head_employee_id`
`GET /sections`	large feed; SCM only needs it for pickers, so `q` search matters more than the full list
`GET /position-levels`	`rank` required
`GET /facilities`	+ `GET /facilities/{id}`
`GET /facility-masters`	facility types
`GET /organization-levels`	office hierarchy levels

Note: `is_stock_location` on a facility is **SCM-owned**. HCM must not send it and must not expect it back.

G. Search / pickers

G.1 `GET /employees/search?q=kumar&limit=20&status=active&office_id=&project_id=&role_id=`

Type-ahead for every "choose a person" box in SCM (delegate an approval, assign a storekeeper, pick a recipient).

- Matches name, employee code and email, prefix + substring.
- **p95 < 150 ms** — it fires on each keystroke (SCM debounces 250 ms).
- Returns the trimmed shape: `id, name, employee_code, status, office.name, primary_position.name`.

G.2 `GET /positions/search?q=&limit=` — same idea for seat pickers.

H. Reconciliation

Webhooks will be missed — a deploy, a network partition, a bug. SCM needs a way to catch up without a full re-pull.

H.1 `GET /changes?since=2026-08-05T09:00:00Z&entities=employees,positions&limit=500`

Every change since a timestamp, ordered by `sequence`.


```
{
  "data": [
    { "sequence": 918233, "occurred_at": "2026-08-05T09:04:11Z",
      "event_type": "employee.deactivated", "entity": "employee", "entity
  ],
  "page": { "next": "…", "total": 37 }
}
```


Ids only (with the event type) is enough — SCM re-reads what it cares about. Retain at least **30 days** of change history.

H.2 `GET /health`


```
{ "data": { "status": "ok", "version": "v1", "server_time": "2026-08-05T1
```


No auth required (or a lightweight key). SCM's admin screen shows this, and SCM's circuit breaker probes it before closing.

I. Plain lists, aggregates, and cross-system joins

Three read patterns that the mirror made trivial and a per-record API does not cover.

I.1 `GET /employees` **— the full filterable list ★**

Not the same thing as search (G.1). SCM's **User Management** screen must page through *every* employee, sorted by name, to answer "who does not have an SCM login yet".

```
GET /employees?
status=active&office_id=&department_id=&project_id=&position_id=&role_id=

&q=&ids=10432,10077&sort=name|employee_code|updated_at&limit=100&offset=0
```

- `ids=` lets SCM ask for an explicit set (its own filters run locally — see I.3).

- `sort` must be server-side and stable; SCM paginates a 5,000-row list off it.
- Same for `GET /positions` (already D.4) and each reference list in §F.

I.2 Aggregates — counts SCM currently gets from `GROUP BY`

SCM's admin screens show "13 positions in this rank", "how many seats per department", "headcount per project". Fetching every row to count it is not acceptable at these volumes.

```
GET /positions/aggregate?
group_by=role|department|office|level&project_id=&status=active
→ { "data": [ { "key": { "id": 4, "name": "Network" }, "position_count":
132, "holder_count": 118 } ] }
```

- `group_by=department` must include an explicit **"no department"** bucket (`"key": null`) — SCM renders one today and a missing bucket loses seats.
- `holder_count` per position is also needed inline on the `Position` object (§4.2).
- Equivalents already requested: §D.2 (ranks per project), §E.1 (`include_headcount`).

I.3 Cross-system joins — the pattern that must not be forgotten ★

Several SCM queries **join a local table to an HCM table in one SQL statement**:

SCM query	Local table	HCM table joined	Becomes
warehouses in an office	`whs warehouses`	`hcm facilities.office_id`	`GET /facilities?office id=&ids only=true` → local `WHERE facility_id IN (…)`
users without a login	`sys user credentials`	`hcm_employees`	I.1 + local set difference
approval stage → seat name	`wf stages.position_id`	`hcm positions.name`	`POST /positions/batch`
notification recipients	`sys notification_rules`	assignments by position	§C.3
document scope filter	`ind *`, `pro_*`	descendant positions	`GET /positions/{id}/descendants?ids_only=true`

The rule: every such join becomes *"ask HCM for a set of ids, then filter locally with `IN (…)`"*. That only works if the `ids_only=true` form exists on **every** filterable list endpoint (employees, positions, facilities, offices, departments,

projects) and can return **up to 20,000 ids in one uncapped response** — a paginated id list defeats the purpose, because SCM cannot run its own query until it holds the whole set.

I.4 `GET /projects?q=&name=`

Lookup a project by name. SCM currently string-matches project names to ids as a workaround; this endpoint plus a real `project.id` on the position object (§4.2) retires that.

---

6. Endpoint summary

#	Method & path	Purpose	Priority
A.1	`GET /employees/{id}`	resolve token subject	**P0**
A.2	`GET /employees/lookup?username=`	login	**P0**
A.2b	`GET /employees/by-code/{code}` + code accepted everywhere	humans use codes, not ids	**P0**
A.3	`GET /employees/{id}/context`	build session in one call	**P0**
A.4	`POST /auth/verify-credentials`	HCM-owned passwords	P2
B.1	`POST /employees/batch`	bulk fetch by ids	**P0**
B.2	`POST /positions/batch`	bulk fetch by ids	**P0**
B.3	`POST /employees/resolve-names`	id → display name	**P0**
C.1	`GET /positions/{id}/ancestors`	approval routing	**P0**
C.2	`GET /positions/{id}/descendants`	row-level scoping	**P0**
C.3	`GET /positions/{id}/holders`	who fills a seat	**P0**
C.4	`GET /employees/{id}/positions`	position switcher	**P0**
C.5	`GET /employees/{id}/manager-chain`	convenience	P2
C.6	`GET /employees/{id}/reports?depth=`	who reports to this person	**P0**
C.7	`GET /positions/{id}/children`	lazy org-chart render	P2
D.1	`GET /roles`	rank list	**P0**

#	Method & path	Purpose	Priority
D.2	`GET /roles?project_id=`	ranks present on a project	**P1**
D.3	`GET /roles/{id}/positions`	ladder preview	**P1**
D.4	`GET /positions?…filters`	position list	**P1**
E.1	`GET /projects[?include_headcount]`	project list	**P1**
E.2	`GET /projects/{id}`	one project	**P1**
E.3	`GET /projects/{id}/employees`	project roster	**P1**
E.4	`GET /projects/{id}/positions`	seats on a project	**P1**
F	reference lists (7 endpoints)	masters/pickers	**P1**
G.1	`GET /employees/search`	type-ahead	**P0**
G.2	`GET /positions/search`	type-ahead	P2
H.1	`GET /changes?since=`	webhook catch-up	**P1**
H.2	`GET /health`	liveness	**P1**
I.1	`GET /employees` (filter + sort + `ids=`)	user-management paging	**P0**
I.2	`GET /positions/aggregate?group_by=`	admin counts	**P1**
I.3	`ids_only=true` on every list endpoint	cross-system joins	**P0**
I.4	`GET /projects?q=`	project lookup by name	P2
W	webhook delivery + §7.6 management endpoints	push changes	**P0**

---

7. Webhooks

7.1 Why

Reads alone are not enough. When someone resigns, SCM must **kill their session and pull their pending approvals immediately** — not on the next cache expiry. Same for a transfer that changes who approves a document, or a seat that gets a new parent.

7.2 Delivery

```
POST https://<scm-host>/api/hcm/webhooks
Content-Type: application/json
X-HCM-Event-Id:    9f1c2e5a-…              (uuid, unique per event)
X-HCM-Event-Type:  employee.deactivated
X-HCM-Delivery-Id: 3a77…                   (unique per attempt; retries
reuse Event-Id, not this)
X-HCM-Timestamp:   1785924247              (unix seconds)
X-HCM-Signature:   sha256=<hex>
X-HCM-Attempt:     1
```

Signature: `HMAC-SHA256(secret, "<X-HCM-Timestamp>.<raw request body>")` , hex. Shared secret, rotatable, with a 24h dual-secret overlap. SCM rejects a request whose timestamp is more than 5 minutes old (replay defence) and one whose signature fails — both with 401.

7.3 Envelope

```
{
  "event_id": "9f1c2e5a-3b6d-4c11-9a2f-77d0e1b4c8aa",
  "event_type": "employee.deactivated",
  "occurred_at": "2026-08-05T09:04:11Z",
  "sequence": 918233,
  "entity": "employee",
  "entity_id": 10432,
  "data": { ...the full current object, same shape as the read API... },
  "changes": { "status": { "from": "active", "to": "inactive" },
               "date_of_exit": { "from": null, "to": "2026-08-05" } },
  "reason": "resignation",
  "actor": { "id": 10001, "name": "HR Admin" }
}
```

- `data` **carries the full post-change object**, not just the diff. SCM stores no HCM rows, so it cannot merge a patch onto anything — it needs the whole record to act on and then discards it.
- `sequence` is a **monotonically increasing integer across all events**. It is how SCM detects a gap (and then calls `GET /changes?since=`) and how it discards an out-of-order redelivery. Not per-entity — one global counter.
- `changes` is advisory; SCM uses it for the audit trail it writes ("HCM deactivated this login at 09:04").
- `reason` is free text but should use a stable vocabulary: `resignation` , `termination` , `retirement` , `transfer` , `promotion` , `restructure` , `correction` .

7.4 Event catalogue

Event	Fires when	What SCM does
Employee		
`employee.created`	new joiner	nothing yet (no SCM login until credentials are issued)
`employee.updated`	name/email/phone/department/section changed	drop cache; **no** change to already-stamped document snapshots
`employee.deactivated`	**resigned / terminated / retired**	**★ revoke every session token for that employee within seconds; block re-login; reassign or flag their pending approvals; notify the workflow admin**
`employee.reactivated`	rejoined / deactivated by mistake	allow login again
`employee.transferred`	office / department / project changed	drop cache; recompute row scope; **re-check in-flight approvals whose next approver is this person**
`employee.manager_changed`	reporting manager changed	drop cached chains; flag any in-flight document whose frozen chain now differs
Position		
`position.created`	new seat	drop rank caches
`position.updated`	name / department / office / rank (`role_id`) changed	drop caches; if `role_id` changed, warn the ladder admin — a configured rung may now be empty
`position.reparented`	★ `parent_position_id` changed	drop chain caches; **flag every in-flight approval whose frozen chain used this seat** (frozen chains are deliberately not rewritten — see §7.5)
`position.deactivated`	seat abolished	drop caches; if a ladder rung points here, raise an admin alert
Assignment		
`position assignment.created`	person moved into a seat	drop caches; new approver becomes live

Event	Fires when	What SCM does
`position assignment.ended`	person vacated a seat	★ if they are the **only** holder of a rung with pending approvals, alert the workflow admin — a vacant rung strands documents
`position assignment.updated`	primary flag / validity changed	drop caches
Org		
`department.created/updated/deactivated`		drop cache
`section.created/updated/deactivated`		drop cache
`office.created/updated/deactivated`		drop cache; office-tree scope recompute
`project.created/updated/closed`		drop cache; a closed project's ladder is frozen
`role.created/updated/deactivated`	rank added or renamed	sync into `rbac_roles` by id
`facility.created/updated/deactivated`		drop cache (`is_stock_location` stays SCM's)
Control		
`resync.required`	HCM did a bulk migration / backfill	SCM flushes every cache and re-reads on demand
`ping`	manual test from HCM's console	SCM 200s and shows it in the admin screen

The starred rows are the ones this integration exists for. `employee.deactivated` must fire **within 30 seconds** of HR marking someone inactive — that is the security requirement driving the whole webhook design.

7.5 What SCM will *not* do on a webhook

Stated so expectations match: once a document is submitted for approval, SCM **freezes** the approver chain onto the approval record. A later reorg does not silently move a purchase order that is already mid-approval. Reorg events therefore

cause SCM to **flag** affected in-flight documents for an admin, not to rewrite them. The same applies to snapshot columns (`raised_by_name` , `office_name`) — historical by design.

7.6 Reliability

- **Ack:** SCM returns `200 {"received": true}` within 5 s. Any non-2xx or timeout = failed delivery.
- **Retries:** at `30 s, 2 m, 10 m, 1 h, 6 h` , then hourly to **24 h**, then dead-letter. Same `event_id` on every attempt; `X-HCM-Attempt` increments.
- **Idempotency:** SCM dedupes on `event_id` . HCM may safely redeliver.
- **Ordering:** not guaranteed by the transport; `sequence` makes it recoverable.
- **HCM-side endpoints SCM needs for operations:**
  - `POST /webhooks/subscriptions` — register URL + events, returns the secret
  - `GET /webhooks/subscriptions` — list
  - `PATCH /webhooks/subscriptions/{id}` — pause / change URL / change events
  - `POST /webhooks/subscriptions/{id}/rotate-secret`
  - `POST /webhooks/test` — send a `ping` to the registered URL
  - `GET /webhooks/deliveries?event_id=&status=failed&since=` — delivery log
  - `POST /webhooks/deliveries/{id}/redeliver` — manual replay
- **Dead letters** are visible in HCM's admin UI and replayable. A silent drop is the one failure mode this design cannot tolerate.

---

8. Non-functional requirements

Endpoint class	p95 latency	Cache TTL SCM will use	Notes
`resolve-names` , `employees/{id}`	100 ms	60 s	highest volume
`search`	150 ms	none	fires per keystroke
`ancestors` , `holders`	200 ms	60 s	blocks a user submitting a document
`descendants`	400 ms	5 min	can be large
batch (≤500 ids)	300 ms	60 s	
`context`	400 ms	per-session	one per login

Endpoint class	p95 latency	Cache TTL SCM will use	Notes
reference lists	500 ms	1 h	
`changes`	1 s	none	background

- **Availability: 99.9%** during business hours. Below that, SCM logins fail — the failure is user-visible and total.
- **Rate limit:** at least **300 req/min** sustained for the SCM key, burst 1000. Return `429` + `Retry-After`, never a silent drop or a slow 200.
- **Maintenance windows** announced 48 h ahead; SCM shows a banner.
- **Payload cap:** 5 MB per response; paginate above that.
- **Timeouts:** SCM's client uses 5 s, 3 retries with exponential backoff, and a circuit breaker that opens after 5 consecutive failures and probes `/health` for 30 s. HCM should assume that behaviour and not treat retries as duplicate work.

---

9. Security

- TLS 1.2+ everywhere, both directions. Certificate pinning is not required but the SCM webhook endpoint must present a valid public certificate.
- IP allow-list on the HCM side for the SCM key, if HCM supports it.
- The SCM key is **read-only**. No endpoint in this document mutates HCM data.
- HCM logs every SCM request with `X-Request-Id`, key id, path and status, retained 90 days.
- Sensitive-field exclusion (§2) is enforced **server-side by HCM**, not by SCM's filter. SCM's scrubber stays as a second line of defence.
- Personal data minimisation: the only personal fields SCM wants are name, employee code, work email, work phone and org placement. Nothing else.

---

10. Migration plan

Phase	Work	Owner
0	Agree this contract; HCM publishes an OpenAPI spec + a sandbox with realistic volumes (~5k employees, ~5k positions)	both
1	HCM ships P0 endpoints (§6) in sandbox; SCM builds an `HcmGateway` behind the current mirror-reading interface	both
2	SCM runs **dual-read**: every mirror query also hits the API, results compared and mismatches logged. No behaviour change. Runs 2 weeks.	SCM

Phase	Work	Owner
3	HCM ships webhooks; SCM implements `POST /api/hcm/webhooks`, session revocation, cache invalidation	both
4	Flip reads to the API, mirror still syncing as a fallback (feature flag)	SCM
5	Drop the 9 `hcm_*` tables, delete `sync.py` / `scheduler.py` / the sync admin screens	SCM

Phase 2 is the one that must not be skipped. The mirror has known drift (positions with cycles, empty `project_id`, missing `head_employee_id`, an empty position-assignments feed that SCM works around by deriving assignments from the `assigned_employee` field inline on positions). Dual-read is how we find out which of those are HCM bugs and which are SCM's compensations before the safety net is removed.

---

11. Open questions for the HCM team

1. `role_id` **coverage** — is every active position guaranteed a `role_id`? A null makes that seat invisible to approval routing.
2. **Project on position vs employee** — `hcm_positions.project_id` is empty in the live feed. Will it be populated, or is employee-level project the only source?
3. `head_employee_id` on departments — absent today. Can it be populated?
4. **Cycles in** `parent_position_id` — will HCM validate and reject on write?
5. **Position-assignments feed** — HCM's own feed is empty and assignments are implied by `assigned_employee` inline on a position. Will C.3/C.4 be backed by real assignment records, including history (`valid_from` / `valid_to`) for the `as_of` parameter?
6. **Passwords** — is A.4 on HCM's roadmap, or does SCM keep owning credentials indefinitely?
7. `sequence` — can HCM guarantee a single global monotonic counter across all entity types, or does SCM need per-entity sequences?
8. **Ex-employee retention** — confirm inactive employees are readable indefinitely (SCM renders their names on documents years later).
9. **Sandbox** — when, and with what data volume?
10. `office_level` — today SCM reads it out of the position's raw payload and it drives row-level scoping. Can it become a typed field with a fixed value set?
11. **Position** `code` **format** — is `<PROFILE-KEY>@<suffix>` a guaranteed convention or an accident? SCM groups seats into job profiles by splitting on `@`.

12. **Uncapped id lists** — will HCM serve `ids_only=true` responses of ~20k ids without pagination? Every local HCM join in SCM depends on it (§I.3).
13. **Production volumes** — how many employees, positions and assignments today, and the expected growth? The latency table in §8 assumes ~5k of each.
14. `employee_code` **uniqueness** — guaranteed unique and never reused, including across leavers and new joiners? SCM wants to key lookups on it (§A.2b).
15. **Reports across multiple seats** — when a person holds two positions, is the union of both subtrees the right answer for "their reports" (§C.6), or does HCM have its own rule?